

Validación de JWT en API Gateway para prevenir token spoofing

Estudiante: Viviana Poblete L.

ADR-005 | Status: Accepted | 2026-05-11

Justificación:

He elegido trabajar sobre la **validación de JWT en API Gateway** porque opera en la capa más temprana de la arquitectura: es la única defensa que actúa *antes* de que cualquier componente del sistema procese la petición.

Las defensas en MCP gateway, output filtering y rate limiting asumen que la identidad del solicitante ya fue verificada correctamente.

Si esa verificación falla, todas las defensas posteriores protegen al usuario equivocado: el agente ejecuta herramientas con los permisos del usuario suplantado, el output filter filtra PII del usuario suplantado, y el rate limiter cuenta requests contra la cuota del usuario suplantado. En términos de threat modeling, una vulnerabilidad en autenticación invalida el modelo de confianza completo del sistema.

Contexto:

El servicio de reportes conversacionales expone un API Gateway que recibe peticiones HTTP de los clientes. La autenticación se delega a un componente Auth/Authz que valida un JWT incluido en el encabezado Authorization. Este JWT es emitido por un Identity Provider (IdP) externo y contiene el user_id y el scope del cliente autenticado.

El threat model identificó que un atacante puede explotar tres vectores sobre el JWT:

- El atacante elimina la firma del token y declara alg=none. Si la librería de validación acepta este valor, el token es aceptado sin verificación criptográfica.
- El atacante toma la clave pública del IdP, cambia el header a HS256 y firma el token con esa clave pública como secret. Librerías mal configuradas aceptan este token como válido.
- Un token legítimamente obtenido, pero ya expirado se reutiliza si el campo exp no se valida.

Restricciones y compromisos:

- Agregar caché del JWKS añade complejidad, pero evita latencia por lookup en cada request.
- La rotación de claves aumenta la seguridad, pero requiere coordinación operacional entre IdP y todos los servicios consumidores.
- Los tres vectores son directamente verificables con pruebas automatizadas, lo que reduce el riesgo operacional de regresión.

Decisión

El API Gateway validará todo JWT entrante usando exclusivamente los algoritmos RS256 o ES256, obteniendo las claves públicas desde el endpoint JWKS del IdP con una caché local de TTL configurable.

Se rechazará cualquier token que declare alg=none, use un algoritmo simétrico (HS*), o cuyo campo exp sea anterior al momento de la validación.

Configuración mínima obligatoria:

```
algorithms_allowed: ["RS256", "ES256"]
verify_exp: true
jwks_uri: "https://idp.interno/oauth2/.well-known/jwks.json"
jwks_cache_ttl_seconds: 300
reject_algorithms: ["none", "HS256", "HS384", "HS512"]
```

Consecuencias

+ Consecuencias positivas	– Consecuencias negativas
Elimina la clase completa de ataques alg=none y RS256→HS256 confusión sin cambios en el cliente.	Agrega latencia en cada request si el endpoint JWKS no está cacheado localmente (mitigable con TTL de caché).
La rotación automática de claves (JWKS) reduce la ventana de explotación si una clave privada se filtra.	Introduce complejidad operacional: rotación de claves requiere coordinación entre el emisor y todos los servicios consumidores.
El rechazo de tokens expirados limita el daño de credenciales robadas a la ventana de validez.	Una ventana de revocación no nula existe hasta que el token expira; JWT sigue siendo stateless (sin blacklist por defecto).
Cobertura testeable: los tres controles tienen pruebas automatizables con curl o scripts CI.	Errores en la config del JWKS pueden dejar el servicio inaccesible hasta que se corrija.

Conexión con el threat model

Esta decisión mitiga directamente la fila de la tabla de amenazas construida en sala:

Componente: API Gateway | **STRIDE:** Spoofing | **OWASP LLM:** LLM06 (Excessive Agency) | **Riesgo:** Alto

La conexión con LLM06 (Excessive Agency) se da porque si el API Gateway acepta un token falsificado, el agente LLM ejecutará herramientas (tx.list, acct. summary) con la identidad y los permisos del usuario suplantado, otorgando al atacante acceso a datos financieros ajenos sin restricción. La corrección en el gateway es la defensa más eficiente porque actúa antes de que cualquier prompt llegue al agente.

Prueba de aceptación:

- curl con token alg=none → respuesta HTTP 401 con body {"error":"invalid_algorithm"}.
- curl con token expirado (exp = now - 60s) → respuesta HTTP 401 con body {"error":"token_expired"}.
- curl con token RS256 válido → respuesta HTTP 200 con datos del usuario correcto únicamente.